NP Notes

1 NP-Complete Proofs

Suppose we have some decision problem X that we would like to prove is NP-complete:

1. **Prove $X \in NP$.** Show that there is we can verify a solution to X in polynomial time.
2. **Reduce a known NP-complete problem to X.** Use some known NP-complete problem Y and show that $Y \leq_P X$. That is, given the input to Y, translate this input to the input of X, and prove that this reduction occurs in polynomial time.
3. **Show that (Input to Y) $\Rightarrow$ (Input to X).** Prove that given an input to Y (the known NP-complete problem) that produces a valid solution, this implies that the reduced input produces a valid solution to X. This is typically the easier direction, as it follows from the reduction we designed.
4. **Show that (Input to X) $\Rightarrow$ (Input to Y).** Prove that given *any* input to X (the original problem) that produces a valid solution, this can be translated into an input to Y that produces a valid solution. (This is typically the more difficult solution, as our reduction was designed to produce a specific instance of X. Now, however, we must consider all instances of X.)

2 NP-Complete Problems

2.1 SAT (+ Variations)

SAT. Given a set of clauses $C_1, ..., C_k$ over a set of variables $X = \{x_1, ..., x_n\}$, does there exist a satisfying truth assignment?

Monotone SAT. Given a SAT formula Φ with no negated variables, is there a way to satisfy Φ with setting at most k variables *true*?

2.2 Independent Set (+ Variations)

Given a graph $G = (V, E)$ and a number k, does G contain an independent set of size at least k?

- A **independent set** $S \subseteq V$ such that no two nodes share an edge (i.e. each $v_i, v_j \in S$ are at least two edges apart).
- A **very independent set** $S \subseteq V$ such that no two nodes share an edge **and** no two nodes share a neighbor (i.e. each $v_i, v_j \in S$ are more than two edges apart).

The generalized problem. Given a graph $G = (V, E)$ and a number k, does G contain some set S of size at least k such that each $v_i, v_j \in S$ are more than n edges apart?

- If $n = 2k + 2$ (i.e. n is even), attach nodes of length $k = \frac{n-2}{2}$ to each vertex in the original graph.
- If $n = 2k+1$ (i.e. n is odd), (1) add a node between each edge in the original graph, and (2) attach nodes of length $k = \frac{n-3}{2}$ to each vertex in the original graph.

2.3 Vertex Cover

Given a graph $G = (V, E)$ and a number k, does G contain a vertex cover of size at most k?

- A **vertex cover** is a set $S \subseteq V$ such that for every $e = (u, v) \in E$, at least one of $u, v \in S$ (i.e. every edge is covered by S).

2.4 Hamiltonian Cycle/Path

Given a directed graph $G = (V, E)$, does it contain a Hamiltonian cycle/path?

- A **Hamiltonian cycle** $C \in G$ is a cycle that visits each node exactly once.
- A **Hamiltonian path** $P \in G$ is a path that visits each node exactly once.

2.5 Traveling Salesman Problem

Given n cities $v_1, ..., v_n$, distances $d(v_i, v_j)$, and a bound D, is there a tour of length at most D?

2.6 Knapsack Problem/Subset Sum

Knapsack Problem. Given a set of n items, each with weight w_i and value v_i, a weight capacity W, and a target value VD, is there a subset of the items with a total weight of at most W and a total value of at least V?

Subset Sum. Given natural numbers $w_1, ..., w_n$, and a target number W, is there a subset of $\{w_1, ..., w_n\}$ that adds up to exactly W?

- This is a special case of the Knapsack Problem with $v_i = w_i$ for each item.

3 NP-Hard Problems on Trees

A problem is **NP-hard** if it is at least as hard as the hardest problems in NP, but does not have to be in NP (i.e. does not have to be verifiable in polynomial time).

Recall that a **tree** is a connected, acyclic, undirected graph. While NP-hard problems are generally difficult on arbitrary graphs, they often become tractable on trees because of their recursive structure. In particular, for any subtree rooted at a node u, the subtree is connected to the rest of the graph only through u, allowing us to solve each subtree independently and then combine those sub-solutions to obtain the overall solution.

3.1 Independent Set

Our objective is to maximize the size of the independent set. Now, note that every tree has at least one leaf (a node of degree 1). This property allows us to design a greedy algorithm.

Consider an edge (u, v) where v is a leaf. Recall that in any independent set, we can include at most one of u or v. If we include v, then the only node excluded is u. However, if we include u, then we can exclude both v and all neighbors of u. Therefore, including the leaf v in our independent set is always at least as good as including u.

Therefore, we claim that there is some optimal independent set that contains every leaf. To prove this, again consider an edge (u, v) with leaf v, and an optimal independent set S. If $v \in S$, then we are done. If $v \notin S$, then $u \in S$, for if neither were in S, we could just add v, contradicting the assumption that S is optimal. Clearly, we can create another optimal independent set S' with $|S'| = |S|$ that swaps v for u. Thus, our claim is indeed true.

We can now use this fact to design the following greedy algorithm:

- Select a leaf v and add it to the independent set S
- Remove v, its neighbor u, and all incident edges from the tree.
- Repeat until the tree has no edges left.

Observe that since each node and edge are processed once, this runs in $O(|V| + |E|) = O(|V|)$ time.